

Iniciamos el lab DevOps React Dashboard

```
kali@Kali: ~/Downloads/Labs/DevOps React Dashboard
Session Actions Edit View Help

W/H/O/A/M/I/_L/A/B/S/_C/O/M/

Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.

[v] Imagen cargada correctamente.
[*] Iniciando laboratorio ...
[v] Laboratorio iniciado correctamente.

[v] Laboratorio desplegado correctamente.

LAB: DEVOPS REACT DASHBOARD
DIFICULTAD: + Fácil (1 punto)

IP del laboratorio (Interna): 172.17.0.2

Ingresa la ROOT flag: 
```

Una vez cargado el lab obtenemos el IP de la máquina a vulnerar, comenzaremos realizando un escaneo de puertos con nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.2 -oN nmap_scan

```
kali@Kali: ~/Downloads/Labs/DevOps React Dashboard
Session Actions Edit View Help

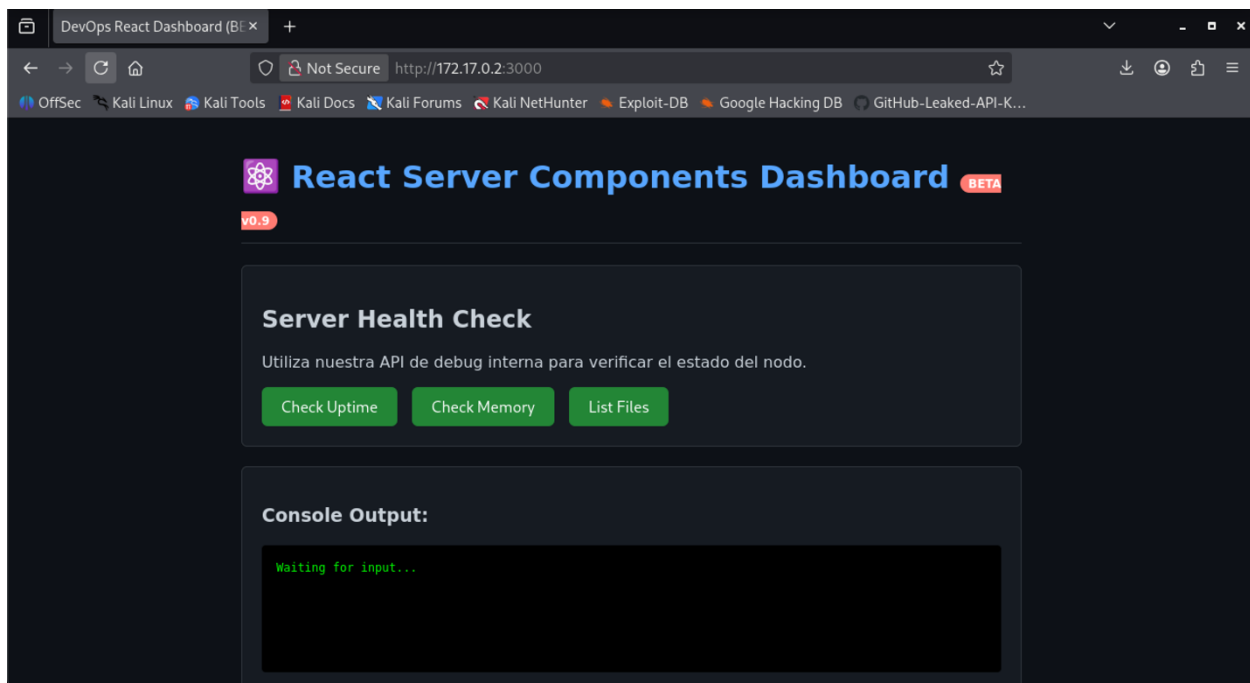
(kali@Kali)~[~/Downloads/Labs/DevOps React Dashboard]
$ nmap -p- -sV -sC -sS -vvv -n -Pn --min-rate=5000 172.17.0.2 -oN nmap_scan
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.99 ( https://nmap.org ) at 2026-06-15 16:54 -0600
NSE: Loaded 158 scripts for scanning.
NSE: Script Pre-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 16:54
Completed NSE at 16:54, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 16:54
Completed NSE at 16:54, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 16:54
Completed NSE at 16:54, 0.00s elapsed
Initiating ARP Ping Scan at 16:54
Scanning 172.17.0.2 [1 port]
Completed ARP Ping Scan at 16:54, 0.05s elapsed (1 total hosts)
Initiating SYN Stealth Scan at 16:54
Scanning 172.17.0.2 [65535 ports]
Discovered open port 3000/tcp on 172.17.0.2
Completed SYN Stealth Scan at 16:54, 0.81s elapsed (65535 total ports)
Initiating Service scan at 16:54
Scanning 1 service on 172.17.0.2
Completed Service scan at 16:55, 11.11s elapsed (1 service on 1 host)
NSE: Script scanning 172.17.0.2.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 16:55
Completed NSE at 16:55, 0.08s elapsed
NSE: Starting runlevel 2 (of 3) scan.
```

```
kali@Kali: ~/Downloads/Labs/DevOps React Dashboard
Session Actions Edit View Help
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 16:55
Completed NSE at 16:55, 0.00s elapsed
Nmap scan report for 172.17.0.2
Host is up, received arp-response (0.0000080s latency).
Scanned at 2026-06-15 16:54:48 CST for 12s
Not shown: 65534 closed tcp ports (reset)
PORT      STATE SERVICE REASON      VERSION
3000/tcp  open  http    syn-ack ttl 64 Node.js Express framework
|_ http-methods:
|_ Supported Methods: GET HEAD POST OPTIONS
|_ http-title: DevOps React Dashboard (BETA)
MAC Address: 02:42:AC:11:00:02 (Unknown)

NSE: Script Post-scanning.
NSE: Starting runlevel 1 (of 3) scan.
Initiating NSE at 16:55
Completed NSE at 16:55, 0.00s elapsed
NSE: Starting runlevel 2 (of 3) scan.
Initiating NSE at 16:55
Completed NSE at 16:55, 0.00s elapsed
NSE: Starting runlevel 3 (of 3) scan.
Initiating NSE at 16:55
Completed NSE at 16:55, 0.00s elapsed
Read data files from: /usr/share/nmap
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 12.60 seconds
Raw packets sent: 65536 (2.884MB) | Rcvd: 65536 (2.621MB)

(kali@Kali)~[~/Downloads/Labs/DevOps React Dashboard]
$
```

Hemos obtenido que tiene el puerto 3000 abierto para http, verifiquemos la página web y su código fuente para ver si podemos obtener alguna información que nos sea de utilidad.



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>DevOps React Dashboard (BETA)</title>
7   <style>
8     body { background-color: #0d1117; color: #c9d1d9; font-family: 'Segoe UI', sans-serif; margin: 0; padding: 20px; }
9     .container { max-width: 800px; margin: 0 auto; }
10    .card { background: #161b22; border: 1px solid #30363d; border-radius: 6px; padding: 20px; margin-bottom: 20px; }
11    h1 { color: #58a6ff; border-bottom: 1px solid #30363d; padding-bottom: 10px; }
12    .status-box { background: #000; color: #0f0; font-family: monospace; padding: 15px; border-radius: 4px; min-height: 100px; white-space: pre-wrap; }
13    button { background: #238636; color: white; border: none; padding: 10px 20px; border-radius: 6px; cursor: pointer; font-size: 16px; margin-right: 10px; }
14    button:hover { background: #2ea043; }
15    .badge { background: #ffb772; color: white; padding: 2px 8px; border-radius: 10px; font-size: 12px; }
16  </style>
17 </head>
18 <body>
19   <div class="container">
20     <h1> React Server Components Dashboard <span class="badge">BETA v0.9</span></h1>
21
22     <div class="card">
23       <h2>Server Health Check</h2>
24       <p>Utiliza nuestra API de debug interna para verificar el estado del nodo.</p>
25       <div>
26         <!-- Llamadas genéricas -->
27         <button onclick="dispatchAction('uptime')">Check Uptime</button>
28         <button onclick="dispatchAction('free -m')">Check Memory</button>
29         <button onclick="dispatchAction('ls -la')">List Files</button>
30       </div>
31     </div>
32
33     <div class="card">
34       <h2>Console Output:</h2>
35       <div id="console-output" class="status-box">Waiting for input...</div>
36     </div>
37   </div>
38
39   <script>
```

Ahora usaremos las herramientas de desarrollo en la parte de red para ver archivos JS/CSS cargados.

The screenshot shows the DevOps React Dashboard in a web browser. The page has a dark theme and contains two main sections: "Server Health Check" and "Console Output". The "Server Health Check" section has three buttons: "Check Uptime", "Check Memory", and "List Files". The "Console Output" section shows a terminal-like interface with the text "Waiting for input...".

The Network DevTools panel is open, showing a list of requests. The first three requests are highlighted in green, indicating successful status codes (200):

Status	Method	Domain	File	Initiator	Type	Transferred	Size	0 ms	5.12 s	10.24 s	15.36 s
304	GET	172.17.0.2:3000	/	document	html	cached	2.77 kB	36 ms			
404	GET	172.17.0.2:3000	favicon.ico	FaviconLoader.sys.mis1...	html	cached	150 B	0 ms			
200	GET	172.17.0.2:3000	sysinfo?cmd=uptime	/:49 (fetch)	json	311 B	76 B		20 ms		
200	GET	172.17.0.2:3000	sysinfo?cmd=free -m	/:49 (fetch)	json	456 B	220 B			12 ms	
304	GET	172.17.0.2:3000	sysinfo?cmd=ls -la	/:49 (fetch)	json	cached	397 B				11 ms

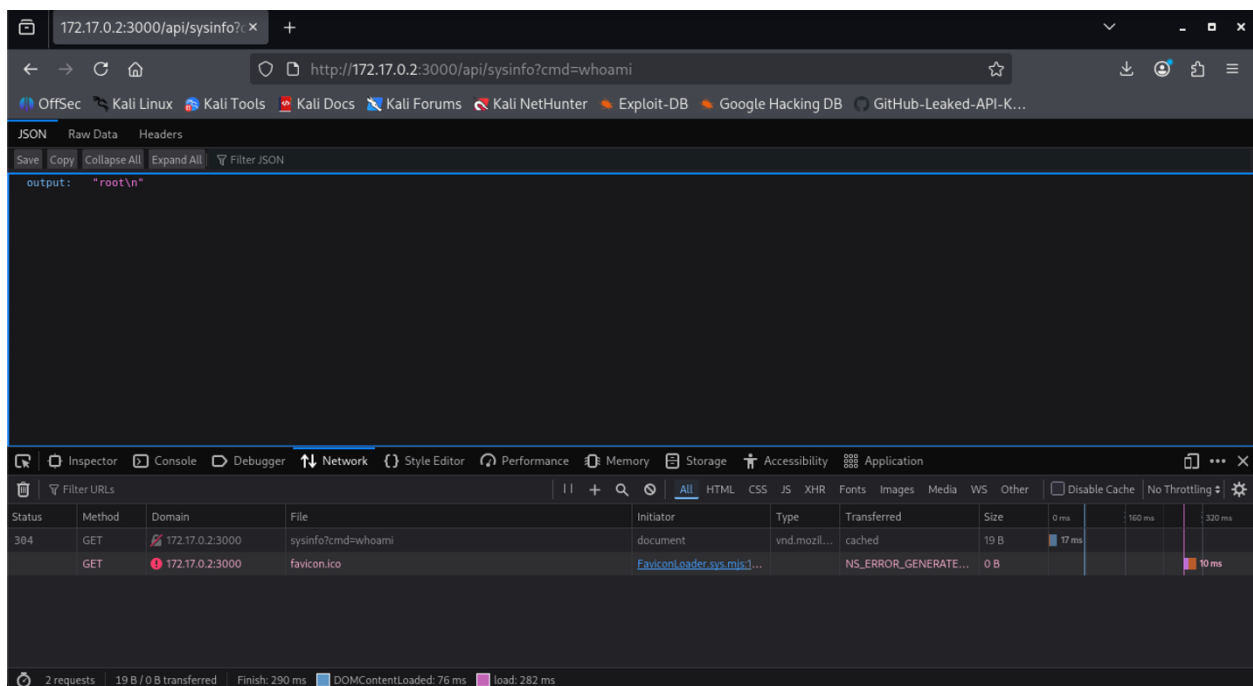
The bottom of the Network panel shows summary statistics: 5 requests, 3.62 kB / 767 B transferred, Finish: 12.19 s, DOMContentLoaded: 116 ms, load: 133 ms.

Si observamos bien cada vez que presionamos uno de los botones manda a ejecutar un comando siguiendo la misma lógica que un web shell.

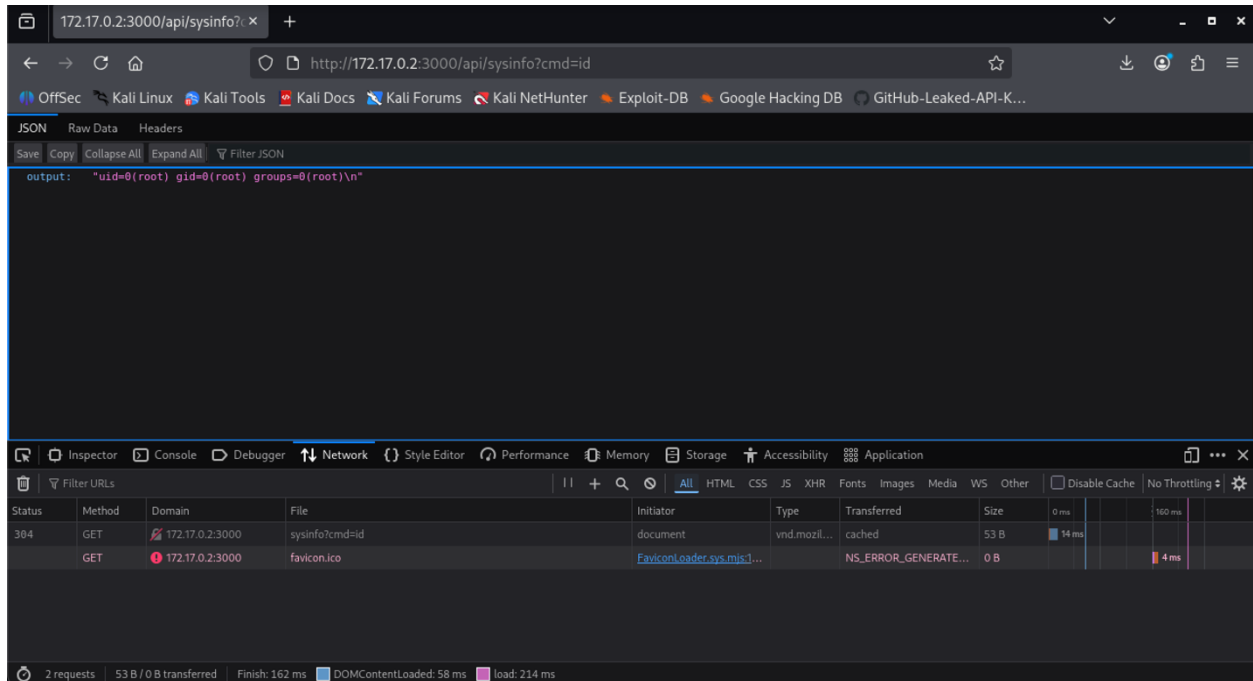
```
23 <h2>Server Health Check</h2>
24 <p>Utiliza nuestra API de debug interna para verificar el estado del nodo.</p>
25 <div>
26 <!-- Llamadas genéricas -->
27 <button onclick="dispatchAction('uptime')">Check Uptime</button>
28 <button onclick="dispatchAction('free -m')">Check Memory</button>
29 <button onclick="dispatchAction('ls -la')">List Files</button>
30 </div>
31 </div>
32
33 <div class="card">
34 <h3>Console Output:</h3>
35 <div id="console-output" class="status-box">Waiting for input...</div>
36 </div>
37 </div>
38
39 <script>
40 // Minified-like logic for cleaner appearance
41 const _API_ENDPOINT = '/api/sysinfo';
42
43 async function dispatchAction(metricType) {
44   const display = document.getElementById('console-output');
45   display.innerText = "Fetching metrics...";
46
47   try {
48     // Fetch logic
49     const req = await fetch(`${_API_ENDPOINT}?cmd=${encodeURIComponent(metricType)}`);
50     if (!req.ok) throw new Error('Network response was not ok');
51     const res = await req.json();
52     display.innerText = res.output || JSON.stringify(res);
53   } catch (error) {
54     console.error('Error fetching metrics:', error);
55     display.innerText = "Error: Unable to reach metrics server.";
56   }
57 }
58 </script>
59 </body>
60 </html>
61
```

También al revisar el código de la página vemos que hay una sección de un script donde hacen referencia a la ruta `/api/sysinfo`.

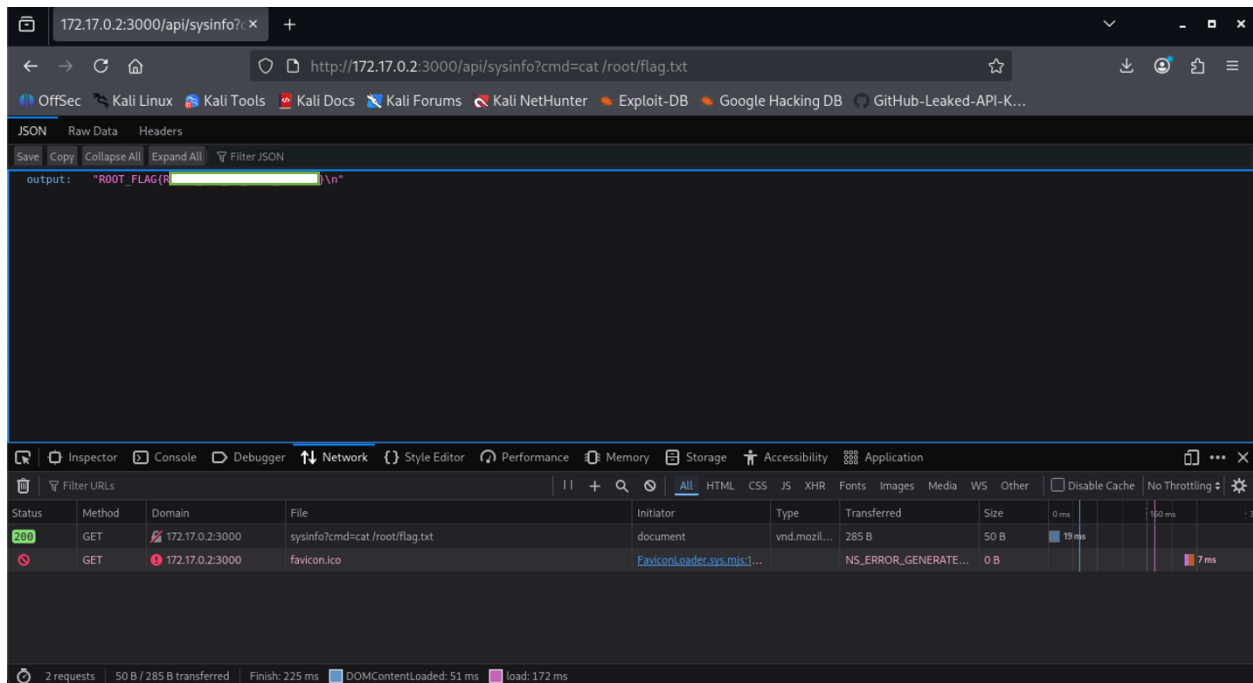
Probemos si esta página está expuesta como una web shell, mandaremos a ejecutar el comando `whoami`



Obtenemos como salida `root`, ahora verifiquemos con el comando `id`



Esto nos confirma que todos los comandos los está ejecutando como root por lo que solo nos queda ir a leer el contenido de la bandera



Una vez obtenida la bandera la verificamos introduciéndola en la terminal donde levantamos el lab

```
kali@Kali: ~/Downloads/Labs/DevOps React Dashboard
Session Actions Edit View Help

W/H/O/A/M/I/-L/A/B/S/.C/O/M/

=====
Bienvenido a WHOAMI-LABS.COM.
Aprende, simula, escala. Sin burocracia.
=====

[v] Imagen cargada correctamente.
[*] Iniciando laboratorio ...
[v] Laboratorio iniciado correctamente.

=====
[v] Laboratorio desplegado correctamente.
=====

LAB: DEVOPS REACT DASHBOARD
DIFICULTAD: * Fácil (1 punto)

=====
IP del laboratorio (Interna): 172.17.0.2
=====

Ingresa la ROOT flag: ROOT_FLAG{R }
¡Felicitaciones hacker! Has conseguido la flag.
¿Quieres eliminar la máquina? (s/n): 
```